

COOP-UAV-S: A Cooperative, Collateral-Damage-Aware Counter-UAS System

A Simulation Framework for Cooperative Interception of Hostile Drone Swarms over Populated Areas

Document type	Full technical report (system design, methods, results)
Subject	Cooperative multi-UAV counter-drone defence of populated areas
System	COOP-UAV-S — pure-Python, ROS 2-shaped, deterministic simulation
Status	v0.3 simulation baseline · 750+ deterministic tests · GPL-3.0
Date	17 June 2026

Scope note. This report documents the simulation environment (Element 2) and command interface (Element 3) of a three-element C-UAS specification. The physical segment (Element 1) is specified but deliberately not built at this stage; it is simulated at high fidelity. No live-fire, procurement, or flight-test activity is in scope. All effector kill probabilities are plausible engineering models, not measured ordnance data.

Abstract

Modern one-way-attack (OWA) drone raids defeat point air defences not with sophistication but with *economics*: they saturate, they mix in signature-identical decoys to exhaust interceptor stocks, and each successful kill drops roughly two hundred kilograms of wreckage onto the neighbourhood being defended. COOP-UAV-S is an open-source simulation framework for studying an AI-assisted, cooperative counter-UAS (C-UAS) system that treats two normally neglected constraints as first-class: (i) interceptors that are individually *slower* than their targets, and (ii) *collateral ground risk* from the debris of a successful kill. The system models a layered sensor network (radar, passive RF, EO/IR, acoustic pickets, forward high-altitude airborne-early-warning sentinels, and terminal onboard seekers) feeding a multi-sensor Kalman/GNN tracker with Bayesian threat-class and decoy belief; a Threat-Evaluation-and-Weapon-Assignment (TEWA) command-and-control loop that reserves cooperative blocking packages using exact Apollonius-circle rendezvous geometry; and a rules-of-engagement layer that runs a Monte-Carlo debris footprint against a SAFE/DANGEROUS/CRITICAL ground-risk raster before authorising any release. The two central claims — that cooperative *geometry* defeats a faster evader where airspeed cannot, and that the predicted debris footprint belongs *inside* the fire decision — are realised in code and exercised by a deterministic, seeded, YAML-driven scenario suite. In a ten-seed Monte-Carlo of the reference residential raid under deliberate saturation, the system holds two hard invariants — **zero wrecks on critical ground** and **zero rounds spent on identified decoys** — while achieving roughly fifty per cent armed-threat attrition. The allocator is a clean software seam: a trained multi-agent (MAPPO) cooperation policy drops in behind the same interface as the classical priority-greedy planner, with every safety gate preserved. The architecture mirrors ROS 2 topics so that migration to ROS 2 / Gazebo replaces two small classes rather than the tactical code.

Keywords — counter-UAS; cooperative pursuit-evasion; Apollonius circle; weapon-target assignment; multi-target tracking; decoy discrimination; ground-risk modelling; rules of engagement; multi-agent reinforcement learning; ROS 2.

Contents

1	Introduction
2	Operational Context and Threat Taxonomy
3	System Architecture
4	Sensing and Perception
5	Command and Control: Threat Evaluation and Weapon Assignment
6	Cooperative Interception Geometry
7	Collateral-Risk-Aware Engagement
8	Learned Cooperative Allocation (MARL)
9	Simulation Methodology and Verification
10	Experimental Results
11	Limitations
12	Related Work
13	Conclusions and Future Work
	References

1 Introduction

The proliferation of cheap, long-range one-way-attack drones has changed the arithmetic of air defence over populated areas. The operational baseline for this work is the Ukrainian theatre of 2022–2026, where raids routinely combine strategic OWA platforms (Shahed-136 / Geran-2 class) with same-signature decoys (Gerbera class), attack densities deliberately exhaust interceptor inventories, and the consequence of *every* successful intercept is roughly two hundred kilograms of falling wreckage. Conventional missile-based defences lose the economic exchange and cannot, in any case, control where the debris lands. COOP-UAV-S studies an alternative: a cooperative team of low-cost kinetic interceptors and a sensor-fused base station, with probabilistic, collateral-damage-aware engagement enforced as a hard constraint rather than an afterthought.

1.1 The two governing ideas

Cooperation beats speed. A lone interceptor slower than its target loses a tail chase — provably; the intercept-triangle equation has no positive root. But a hostile OWA is mission-bound to a predictable corridor toward its target, so a *team* can post blockers at corridor points they can reach first and let the target fly into the engagement. The C2 reserves blocking packages for exactly the targets that outrun the fleet, placing posts with closed-form Apollonius-circle rendezvous geometry. It is the geometry, not the airspeed, that wins.

Where the wreck falls is part of the fire decision. The defended area is rasterised into SAFE / DANGEROUS / CRITICAL ground. Before any munition release the C2 runs a Monte-Carlo debris footprint of the *predicted* kill against that map and clears, holds, or denies the shot — including a ‘now-or-never’ rule that authorises an imperfect shot when the target’s own trajectory guarantees every later shot is worse. Net-gun kills drop wreckage nearly straight down; projectile kills throw it forward — and the rules of engagement feel the difference.

1.2 Contributions

- A complete, deterministic, message-driven C-UAS battle simulation spanning threats, layered sensing, multi-sensor fusion, TEWA command-and-control, cooperative interceptor agents, and a ground-truth engagement adjudicator.
- An exact Apollonius-circle formulation of cooperative blocker placement that defeats a faster, corridor-bound evader with slower pursuers, in relay.
- A debris-footprint rules-of-engagement layer that integrates predicted collateral ground cost directly into weapon-release authorisation, with clearance / hold / deny semantics.
- Forward high-altitude combat-air-patrol (CAP) sentinels carrying a look-down airborne early-warning radar, cutting diving-jet acquisition latency by roughly 36×.
- A learned multi-agent (MAPPO) cooperation policy that drops in behind the classical allocator seam without altering any downstream safety gate.
- A ROS 2-shaped architecture and a YAML-as-experiment methodology that make every run reproducible and the migration path concrete.

The remainder of the report follows the data path: operational context and threat taxonomy (§2), the architecture (§3), sensing and perception (§4), command and control (§5), the two innovation pillars — cooperative geometry (§6) and collateral-aware engagement (§7) — the learned allocator (§8), methodology and verification (§9), results (§10), limitations (§11), related work (§12), and conclusions (§13).

2 Operational Context and Threat Taxonomy

The system is designed around operationally-validated drone threat classes. The defining tactical patterns are altitude switching (low terrain-masking ingress versus high-altitude terminal dive), saturation (answered with low cost-per-shot kinetic effectors rather than missile economics), decoy integration (answered with multi-modal Bayesian discrimination), and jam-immune fibre-optic FPV guidance (answered kinetically). The nominal operating envelope is nocturnal, all-weather, −25 °C to +45 °C, winds to 20 m/s, and altitudes from 50 m to 5 km above ground level.

Class	Exemplar	Mass	Speed	Altitude (AGL)	Behaviour
A — Strategic OWA	Shahed-136 / Geran-2	~200 kg	50–65 m/s	50 m – 5 km	Swarm saturation, decoy mixing, terminal dive
A+ — Jet OWA	Geran-3 / Shahed-238	~200 kg	~100 m/s	2–5 km	High-speed dive; very low intercept window
B — Tactical FPV	Quadcopter kamikaze	1–5 kg	30–40 m/s	0–200 m	Agile, fibre-optic guided (jam-resistant)
C — Loitering munition	Lancet-3	12 kg	~80 m/s	50–500 m	AI terminal seeker, precision strike
D — Decoy	Gerbera	~18 kg	as class A	as class A	Identical RF/radar signature, no warhead — exists to exhaust interceptor stocks

Table 1. Threat taxonomy. Class D decoys share the class A radar and RF signature and flight profile by design: perception genuinely cannot separate them until a discriminating sensor earns it.

The decoy is the structurally interesting class. A Gerbera carries no warhead; its sole purpose is to draw an interceptor. Because it is built to be indistinguishable from a class-A OWA on radar and RF, a defender that fires on radar tracks alone spends its magazine on empty airframes — which is precisely the adversary’s objective. Discrimination therefore cannot be a pre-filter; it must be a belief that the system accumulates over time from the sensors capable of telling the two apart (EO/IR identification, acoustic engine cues, kinematic consistency), and it must be fused with the cost of being wrong.

3 System Architecture

3.1 Three-element system definition

The governing System Requirements Specification partitions the system into three elements. **E1**, the physical segment (interceptor UAVs, ground stations, fixed sensors, anti-air turrets), is specified but not built at this stage — its requirements exist as the fidelity reference. **E2**, the simulation environment, is a high-fidelity digital twin of E1, its software, its communications, the threats, and the physical world. **E3**, the command interface, is the human-facing 3D real-time C2 console plus its orchestration agent. The binding principle is that E3 — and the humans behind it — cannot functionally distinguish the simulator from the real system except through explicitly evaluation-only ground-truth channels. Every physical requirement (PHY-*) has a corresponding simulation requirement (SIM-*) that recreates it; replacing E2 with real assets requires no change to E3 beyond disabling the evaluation channel.

3.2 Design philosophy

Three decisions shape everything else. First, a **custom Python simulation with ROS 2-shaped seams**: every software component is a node on a publish/subscribe message bus with typed dataclass messages — a one-to-one image of ROS 2 nodes, topics, and .msg files — so migration to ROS 2 + Gazebo/PX4 replaces two small classes and the sim-side plugins while the tactical code is untouched. Second, **probabilistic engagement, not ballistics**: effectors expose a kill-probability *envelope* over range, off-axis angle and closing speed, and kills produce sampled debris footprints — the right fidelity for studying cooperation geometry and collateral risk, and swappable for higher fidelity later. Third, **ground truth is quarantined**: only explicitly sim-side components (sensors and the engagement adjudicator) may read the world’s true state, exactly like Gazebo plugins; perception, C2 and the interceptor agents see nothing but messages. Decoys are therefore genuinely indistinguishable until a sensor earns the discrimination.

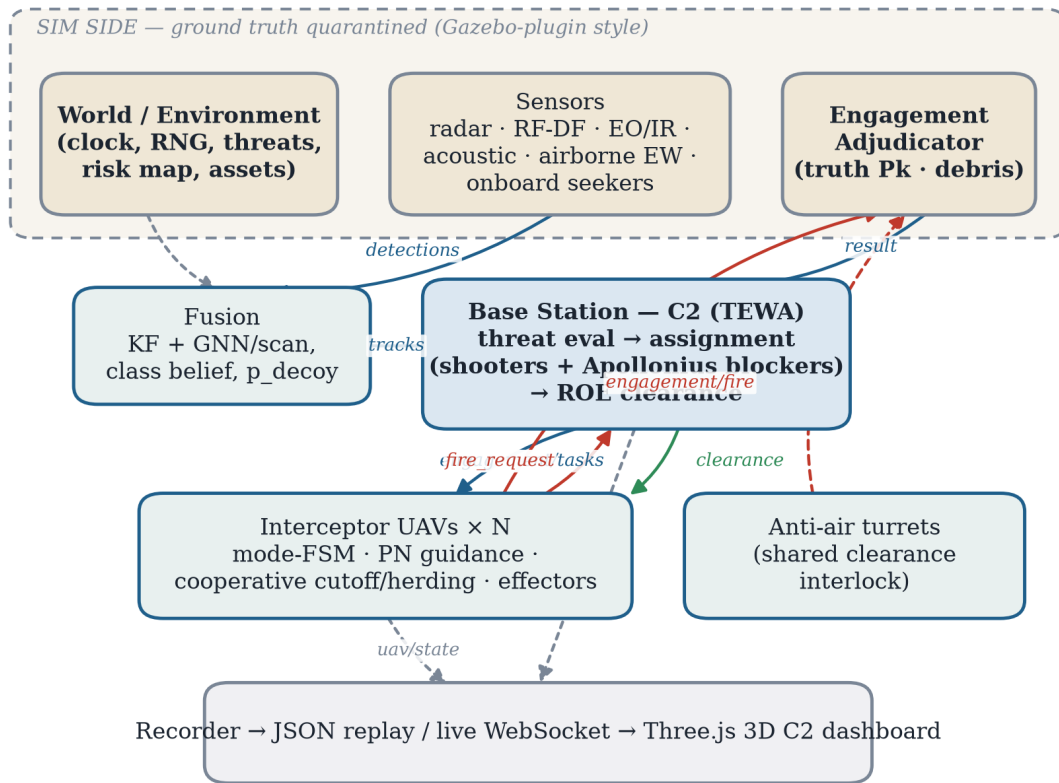


Figure 1. Component graph and publish/subscribe dataflow. Sim-side nodes (beige) own ground truth; tactical nodes (blue) act only on messages. The fire request / clearance handshake at the C2 is the human-on-the-loop seam.

3.3 Topic contract

The bus topics are the future ROS 2 message interfaces. The full contract is small enough to state in one table.

Topic	Message	Producer → Consumer
detections	Detection	all sensors → Fusion
tracks	TrackArray	Fusion → C2, UAVs, Recorder
uav/state	UavState	each UAV → C2, peer UAVs, Recorder
engagement/tasks	list[EngagementTask]	C2 → UAVs
engagement/fire_request	FireRequest	shooter UAV → C2
engagement/clearance	FireClearance	C2 → shooter UAV
engagement/fire	FireRequest	shooter UAV → Adjudicator
engagement/result	EngagementResult	Adjudicator → C2

Table 2. The topic contract. Each row maps directly onto a ROS 2 topic and message type.

4 Sensing and Perception

4.1 Layered, mutually-compensating sensing

No single sensor solves the problem; each is deliberately imperfect and the layer is designed so that the weaknesses are complementary. All sensors emit the same `Detection` message with a full 3×3 covariance, so bearing-only geometry is encoded as an anisotropic covariance rather than a special case and the tracker needs no per-sensor logic.

Sensor	Strength	Weakness modelled
Radar	Long range, Doppler	R■ Pd falloff; radar horizon hides low FPVs
Passive RF-DF	Very long range, signature hash	Bearing-only; decoys share the OWA signature by design
EO/IR towers	The decoy discriminator	Short range; ID quality ramps with proximity
Acoustic pickets	Hear below the radar horizon	Short range, coarse bearing
Airborne EW (sentinel)	Look-down on high divers	Forward deployed; on-station endurance limits
Onboard seeker	Terminal accuracy + close ID	Only where an interceptor already is

Table 3. The sensor layer. The radar and RF set provide volume and persistence; EO/IR, acoustic and onboard seekers provide the discrimination the volume sensors cannot.

4.2 Tracking, fusion and classification

Perception is a three-stage pipeline. **Tracking** maintains a six-state constant-velocity Kalman filter per object, with separate filter-time and measurement-time bookkeeping so that coasting tracks are pruned correctly. **Fusion** performs per-scan global-nearest-neighbour association — the Hungarian algorithm over Mahalanobis-gated costs — and fuses sensors sequentially in precision order, so that radar seeds tracks rather than RF bearing blobs. **Classification** maintains a Bayesian class belief per track from sensor likelihoods and RF signature evidence, deliberately joint over {OWA, decoy}; kinematic consistency is blended idempotently at readout rather than accumulated, because double-counting the same kinematic evidence every cycle would saturate the posterior. The output that matters downstream is `p_decoy`, the probability that a track is a warhead-less decoy.

Why decoy belief must be a posterior, not a flag. Because the adversary engineers the decoy to match the OWA on the volume sensors, the only way to lower `p_decoy` is to earn discriminating evidence — an EO/IR identification at closing range, an acoustic engine-type cue, or kinematic inconsistency with a warhead profile. The allocator then trades that belief against the cost of a wrong call: firing on a true OWA you mistook for a decoy is a leak; firing on a decoy you mistook for an OWA is a wasted interceptor — the adversary’s goal.

5 Command and Control: Threat Evaluation and Weapon Assignment

The base station runs a Threat-Evaluation-and-Weapon-Assignment (TEWA) loop at 1 Hz in three stages.

5.1 Threat evaluation

Each track is propagated in the horizontal plane to predict impact against the protected-asset list. Its threat score combines lethality ($1 - p_decoy$), urgency (time-to-impact), asset value, and a ground-zone factor — so a likely-decoy track heading nowhere important scores far below a likely-OWA track diving on a hospital.

5.2 Assignment

The allocator is priority-greedy over threat-ordered tracks. Each track receives exactly the package it needs: a shooter if the fleet can catch it, or a shooter *plus reserved blockers* if it outruns the fleet. A budget rule prevents support reservation from starving queued tracks under saturation; an incumbent discount ($0.7\times$) suppresses jitter-driven shooter swaps. Tracks whose decoy probability exceeds 0.85 are assigned nothing — spending interceptors on Gerberas is the enemy’s actual objective. The whole allocator sits behind a single function-call seam (§8), with any allocator exception falling back to the classical planner for that cycle so that a misbehaving policy can never freeze tasking.

5.3 Fire authorisation

Every release costs a Monte-Carlo debris footprint against the risk map (§7), evaluated out-of-band — immediately, not at the 1 Hz planning rate — because an engagement window against a 55 m/s target lasts only seconds. The rules of engagement return one of five verdicts:

- **AUTHORIZED (geometry_safe)** — footprint cost under the base thresholds.
- **AUTHORIZED (now_or_never)** — above the base threshold, but the footprint cost is minimal over the target's *predicted* path: it is flying into the city, so holding only moves the debris onto worse ground.
- **AUTHORIZED (last_resort)** — impact imminent on a high-value asset.
- **HOLD** — the geometry can still improve; wait.
- **DENIED** — decoy-grade target, or unsafe geometry; disengage.

There is no release without an explicit clearance message. The fire-request / clearance handshake is precisely where a human-on-the-loop operator plugs into the message flow; all engagement authority traces to a human action or a human-pre-approved ROE rule.

6 Cooperative Interception Geometry

The first innovation pillar answers a hard kinematic fact: a propeller interceptor at ~ 80 m/s cannot run down a 100 m/s jet OWA, and frequently cannot catch even a 60 m/s OWA from a tail-chase start. The intercept-triangle solution — the quadratic whose positive root is the time to interception — simply has no root when the geometry is unfavourable. That absence of a root is the trigger for cooperation.

6.1 Apollonius-circle blocker placement

For a pursuer at $\mathbf{P}$ with speed v_p and an evader at $\mathbf{T}$ with speed v_T , the locus of points the two reach simultaneously satisfies $|\mathbf{X}-\mathbf{P}| = k|\mathbf{X}-\mathbf{T}|$ with $k = v_p/v_T$. For $k < 1$ (a slower pursuer) this locus is the **Apollonius circle**, with centre $\mathbf{C} = (\mathbf{P} - k^2\mathbf{T})/(1-k^2)$ and radius $R = k|\mathbf{P}-\mathbf{T}|/(1-k^2)$; inside the disk the pursuer arrives first. A lone slower pursuer cannot capture a free evader — but the OWA is not free. It is mission-bound to a corridor toward its target, so the defender posts a blocker at the point where that corridor *first enters* the reachable disk. The interceptor arrives there first and meets the target head-on. The C2 reserves these blocking packages only for targets that outrun the fleet, and chains them in relay so that successive interceptors hand off containment.

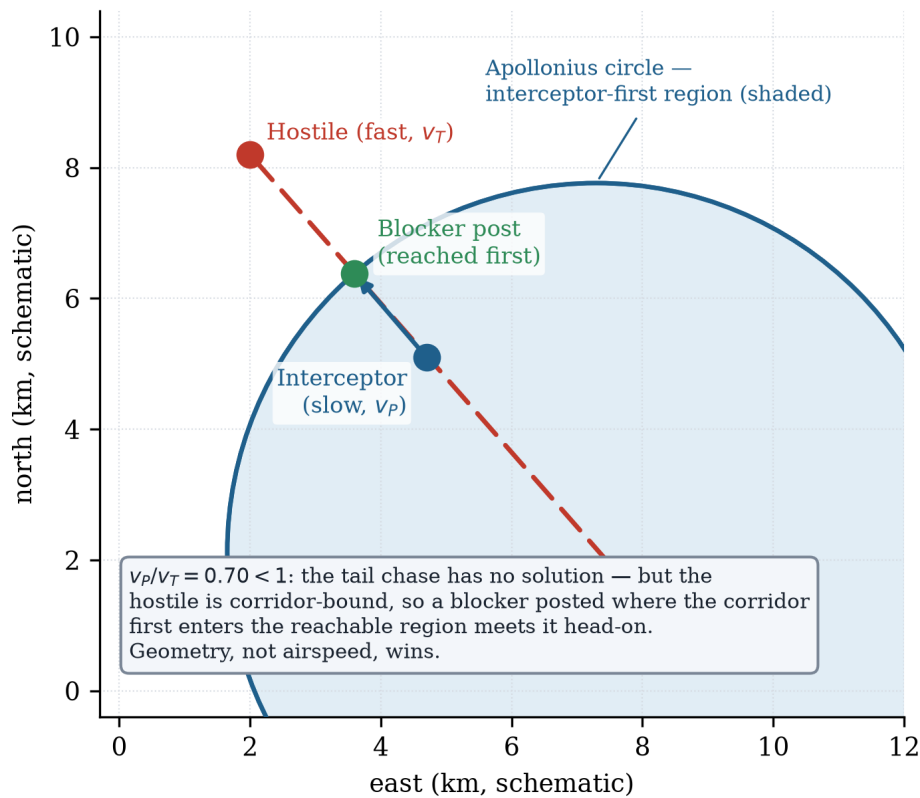


Figure 2. Apollonius cooperative-interception geometry for a slower pursuer ($v_p/v_T = 0.70$). The shaded disk is the region the interceptor reaches before the corridor-bound hostile; the blocker is posted where the predicted corridor first enters it.

The implementation (`mc/apollonius.py`) uses the exact closed-form rendezvous, replacing an earlier time-stepping search, and extends to the game-theoretic containment arc and an escape-set (safe-fraction) area objective for manoeuvring evaders. The interceptor agents themselves run a mode finite-state machine (idle, pursuit, engage, blocking, herding, return-to-base) with proportional-navigation-style guidance; blocking-versus-herding is derived downstream from the target-versus-shooter speed comparison rather than chosen explicitly.

6.2 Forward CAP sentinels

Cooperative geometry only helps if the target is seen early enough to post the blocker. The high-altitude diving jet OWA is the worst case: it appears late to the ground sensor set and leaves a vanishing intercept window. The system answers with forward combat-air-patrol sentinels — unarmed patrol UAVs standing on station above the ground radar’s envelope and forward of the defended area, carrying a look-down airborne early-warning radar. They publish ordinary detections and, in the high-diver scenario, cut mean acquisition latency from 2.15 s to 0.06 s — roughly a 36× improvement (Figure 3).

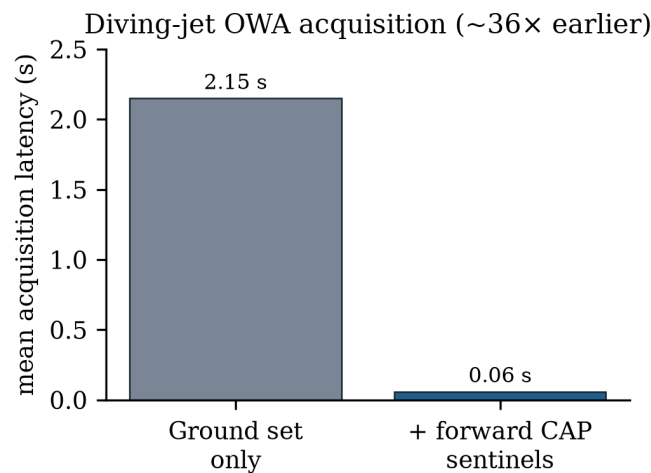


Figure 3. Diving-jet OWA mean acquisition latency in the high-diver scenario, ground sensor set versus the set augmented with forward CAP sentinels.

7 Collateral-Risk-Aware Engagement

7.1 The ground-risk raster

The defended area is rasterised into a SAFE / DANGEROUS / CRITICAL grid with zone weights 0.02 / 1.0 / 25.0. The default for unknown ground in a residential area is DANGEROUS, not SAFE: the system assumes civilians are present unless the map says otherwise. With building-typed urban inputs the raster is painted from building kinds in precedence order — a SAFE base, DANGEROUS residential and commercial buffers, SAFE parks, water and industrial land, and CRITICAL hospital, school and dense-residential buffers — so that red means civilians are certainly under the debris, yellow possibly, and green not at all. The design is SORA/JARUS-inspired and follows the urban ground-risk-map literature.

7.2 The debris model

A kill produces a sampled ballistic debris footprint. The model is mechanism-dependent: a net-gun kill retains only ~0.15 of the target’s horizontal velocity (the wreck drops nearly straight down), whereas a projectile-gun kill retains ~0.65 (the wreck is thrown forward), with terminal-velocity fall time and altitude-growing dispersion. The same model is used twice: *predictively* inside the rules of engagement to cost a candidate shot, and *generatively* by the adjudicator when a kill actually happens. This is why the choice of effector is itself part of the collateral calculus, and why the ROE can prefer a net-gun shot that drops the wreck on safe ground over a projectile shot that throws it onto a school.

7.3 Interceptable debris

A kill spawns a falling debris object stepped by the world, publishing its predicted impact point and zone. The C2 turns any non-SAFE-bound debris into intercept tasks for projectile UAVs and turrets — CRITICAL before DANGEROUS — and credits the averted zone cost when a debris intercept succeeds; SAFE-bound debris is left to fall. The collateral problem is thus addressed both before the shot (footprint-gated authorisation) and after it (active debris interception).

8 Learned Cooperative Allocation (MARL)

The weapon-target allocator is a clean software seam: the base station calls it through one hook, and a trained multi-agent policy can replace the classical priority-greedy planner behind the *same* interface. The policy decides which threats to commit which interceptors to, and in what role; everything downstream — guidance, the debris-footprint ROE, fire-control, and the authorisation chain — is the unchanged, trusted stack. Critically, the reconciliation step routes the final shooter pick through the classical best-shooter selector verbatim and re-applies the denied-track, debris-effector and availability gates, so the Pk-aware, incumbent-discounted choice the ROE geometry assumes stays authoritative even if the policy mis-commits.

8.1 Problem formulation

The agents are the interceptors. A single parameter-shared actor runs per agent on an ego-centric, 176-dimensional observation (own state; the top-K = 6 threat tracks by score, with relative kinematics, threat score, time-to-impact, `p_decoy`, impact zone, and caller-owned catchable / incumbent / denied / debris / valid flags; the nearest M = 4 teammates; and a small global block), so the policy generalises across platforms and fleet sizes and can deploy decentrally. A centralised critic sees the joint observation in training only (the CTDE / MAPPO pattern). The action is one masked categorical over $1 + 2K = 13$ choices — idle, shoot one of the top-K, or block one of them — with the mask forbidding empty slots, denied tracks and net-on-debris so that any sampled action reconciles to a valid task. The reward is a shared team outcome (positive for armed kills; heavily negative for armed leakers, the real defence failure) plus zone-weighted debris and decoy penalties, per-agent ammunition and task-churn penalties, and a potential-based safety-shaping term that is policy-invariant.

8.2 Training and deployment

Because the simulation is pure-Python and CPU-bound, training throughput scales with worker *processes*, not threads, and needs no GPU; the env step, not the network, is the bottleneck for a small WTA policy. Each episode draws a fresh randomised raid, so the policy sees a distribution of raids rather than one script. Deployment is a two-line scenario change pointing the base station at a checkpoint, and an A/B evaluation harness compares the learned allocator against the classical baseline over a seed sweep. The training and inference pipeline is complete and verified; the shipped default behaviour remains the classical planner, with the learned policy opt-in — a policy that decisively beats a well-tuned classical baseline requires a full training run, which is honestly noted as outstanding.

9 Simulation Methodology and Verification

The simulation is a deterministic fixed-step world (default 20 Hz) with per-node rates, driven by a single seeded random-number generator so that every run is exactly reproducible — a property enforced by a dedicated determinism test. Scenarios are pure YAML: each file fully describes a battle — map, risk zones, sensor laydown, fleet, raid composition, and ROE thresholds — so that experiments are data, not code. The engagement adjudicator referees every fire event against ground truth, rolling the true kill probability and sampling the debris footprint, and is one of only two components permitted to read the world’s true state.

Three reference scenarios anchor the experiments. The **residential raid** is the canonical case: a 12×12 km area with two protected assets and a nine-drone three-wave mix of OWAs, decoys, FPVs and one jet OWA. The **high-diver raid** isolates the high-altitude diving-jet problem and the forward CAP sentinel response. The **urban raid** is a large procedurally-generated city with twenty interceptors, ten sentinels, line-of-sight occlusion and live debris. The

framework ships with a suite of 750+ deterministic tests, including an end-to-end raid, and an optional higher-fidelity software-in-the-loop mode that runs the tactical stack on a virtual flight controller (EKF, mixers, built-in-test fault monitors) over a modelled datalink.

Scenario	What it exercises
residential_raid	Reference raid: 12×12 km, two assets, 9-drone three-wave OWA/decoy/FPV/jet mix
high_diver_raid	High-altitude diving-jet problem + forward CAP sentinels with look-down EW radar
urban_raid	Large procedural city: 20 interceptors, 10 sentinels, occlusion, live debris

Table 4. The reference scenario suite. Scenarios are YAML data; copying and editing one defines a new experiment with no code change.

10 Experimental Results

The headline evaluation is a ten-seed Monte-Carlo of the reference residential raid — nine threats including two decoys, defended by six gun and two net interceptors — run under deliberate saturation. A representative single run resolves as six kills, three leakers of which one is armed, and wreckage falling on one SAFE and two DANGEROUS cells with none on CRITICAL ground (Figure 4a–b). Across all ten seeds the system holds two hard invariants.

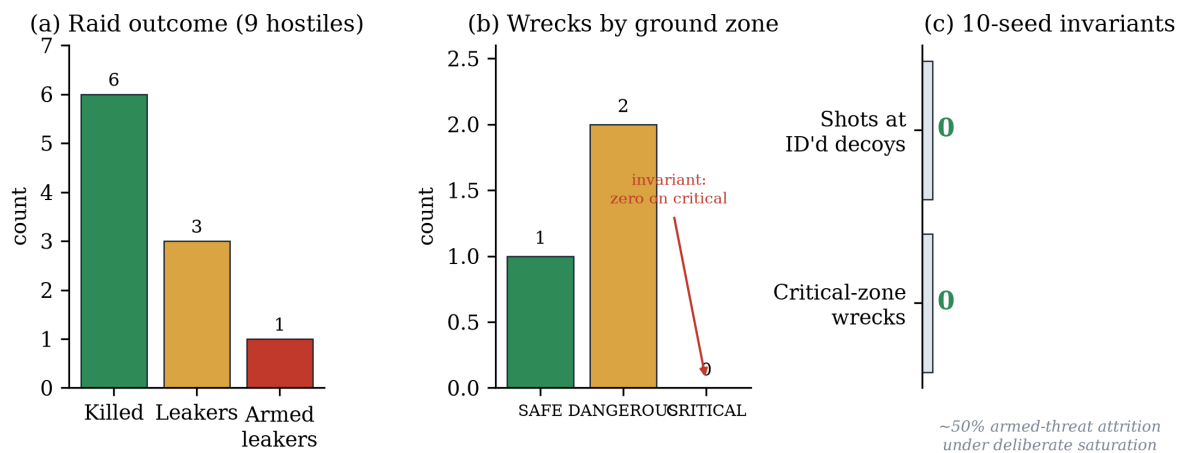


Figure 4. Reference residential-raid results. (a) Representative single-run outcome over nine hostiles. (b) Debris by ground zone — the ROE invariant of zero critical-zone wrecks. (c) The two ten-seed invariants, with overall armed-threat attrition.

Verified ten-seed invariants (reference residential raid)

- **Zero critical-zone wrecks** across all seeds — the collateral ROE invariant holds under saturation.
- **Zero shots at identified decoys** across all seeds — interceptor economics are preserved against the Gerbera exhaustion tactic.
- **~50% overall armed-threat attrition**, with a class structure that matches reality: strategic OWAs and FPVs engaged effectively; the 100 m/s jet OWA documented as beyond the propeller-interceptor tier (motivating the fast-interceptor tier and the forward CAP sentinels).

These two invariants are the project’s central empirical claim: under a saturation raid engineered to exhaust the magazine and to force collateral damage, a cooperative, collateral-aware C2 spends nothing on decoys and drops nothing on critical ground, while still attriting roughly half of the armed threats. The separately measured 36× reduction in diving-jet acquisition latency from the forward CAP sentinels (Figure 3) shows that the remaining hard case — the fast diver — is a sensing-and-cooperation problem, which is the gap the learned allocator and the fast-interceptor tier are designed to close.

11 Limitations

The framework is explicit about the fidelity boundaries of the current baseline.

- **Effector kill probabilities are plausible engineering models, not measured data.** No public Pk data exists for any C-UAS interceptor, so the engagement envelopes are inventions calibrated for qualitative realism, not validated ordnance performance.
- **The tracker is constant-velocity** and therefore lags manoeuvres; this is mitigated by the onboard terminal seeker but motivates the planned interacting-multiple-model upgrade.
- **The battery model is a linear drain** with no recovery or rearm cycle in the tactical core: interceptors return to base and stay there. (A higher-fidelity powertrain and charging-station cycle exists in the SITL layer.)
- **The learned policy is not yet a proven win.** The pipeline is complete and verified, but demonstrating a policy that beats the well-tuned classical baseline needs a full training run; the shipped behaviour is the classical planner.
- **The three-class risk raster is a coarse proxy** for true population exposure; time-of-day population layers and casualty-expectation units are planned.

12 Related Work

COOP-UAV-S draws on six largely separate literatures and integrates them into one closed loop. **Cooperative pursuit-evasion** provides the geometric backbone: the multiple-pursuer differential-game series of Garcia, Casbeer, Von Moll and Pachter [2,3], the faster-evader treatments of Wang et al. [4] and the Apollonius-with-obstacles work [5], and the area-optimal safe-reachable-set formulation [6] that motivates the escape-set objective; Isaacs [1] is the foundational text. **Weapon-target assignment and TEWA** supply the allocation layer — the modern WTA survey [7], consensus-based decentralised auctions (CBBA) [8] for the link-degraded case, and the Hungarian assignment [9] used in fusion. **Multi-target tracking** contributes the filtering and association canon — MHT [10], the GM-PHD and labelled-RFS filters [11,12], JPDA/IMM [13] for the planned manoeuvre upgrade, and the Stone Soup framework [14] as a reuse target. **Decoy and drone discrimination** rests on radar micro-Doppler [15] and RF-fingerprinting [16] research, against open-source intelligence on the Gerbera decoy [17]. **Guidance** follows proportional-navigation theory [18] and its multirotor and net-capture adaptations, including the CTU-MRS MBZIRC drone-capture work [19]. **Ground-risk modelling** follows JARUS SORA [20] and the urban ground-risk-map and impact-footprint literature [21,22]. Finally, the **multi-agent reinforcement learning** layer uses the MADDPG and MAPPO baselines [23], and the ROS 2 / Aerostack2 / Gazebo stack [24] defines the migration path. To our knowledge the specific combination — cooperative Apollonius blocking in relay, with debris-footprint-gated weapon release against a civilian-presence raster, evaluated under decoy-saturated raids — is not addressed as a closed loop in the open literature.

13 Conclusions and Future Work

COOP-UAV-S demonstrates, in a deterministic and fully reproducible simulation, that the two constraints usually treated as afterthoughts in counter-drone work — interceptors slower than their targets, and the collateral cost of where the wreck falls — can be made first-class and still yield an effective defence. Cooperative Apollonius-circle blocker placement converts a kinematically hopeless tail chase into a head-on engagement against a corridor-bound evader; a debris-footprint rules-of-engagement layer keeps wreckage off critical ground; and a decoy-aware allocator refuses to spend the magazine on the adversary's bait. Under a saturation raid the system holds both hard invariants — zero critical-zone wrecks and zero shots at identified decoys — at roughly fifty per cent armed-threat attrition.

The near-term roadmap closes the loop the current ROE leaves open: **intercept-point optimisation** — choosing *where* along the corridor to shoot by minimising expected debris cost subject to a Pk floor, unifying kill-box selection, herding and ROE — is the most defensible novelty and addresses an open-literature gap. Other planned work includes interacting-multiple-model tracking for dive manoeuvres, decentralised CBBA allocation for link-degraded operation, population-density risk maps with casualty-expectation units, a decoy-ratio sweep study, and a full MAPPO training run to test whether the learned policy can beat the classical baseline. The longer-term path is fidelity and migration: a ROS 2 port that regenerates the message contract as .msg files, a Gazebo / PX4 software-in-the-loop slice for a few vehicles, and richer sensor and effector flyout models. Because the architecture was built ROS 2-shaped from the start, that

migration replaces the bus and node classes — not the tactical code this report describes.

References

- [1] R. Isaacs, *Differential Games: A Mathematical Theory with Applications to Warfare and Pursuit, Control and Optimization*. Wiley, 1965 (Dover reprint, 1999).
- [2] E. Garcia, D. W. Casbeer, A. Von Moll, and M. Pachter, “Multiple Pursuer Multiple Evader Differential Games,” *IEEE Trans. Automatic Control*, 2020.
- [3] A. Von Moll, D. Casbeer, E. Garcia, D. Milutinović, and M. Pachter, “Cooperative Pursuit by Multiple Pursuers of a Single Evader,” and “Multiple-Pursuer, Single-Evader Border Defense Differential Game,” *J. Aerospace Information Systems*.
- [4] X. Wang et al., “Cooperative Hunting Strategy with a Superior Evader Based on Differential Game,” *Complexity*, 2022.
- [5] “Collaborative pursuit-evasion game of multi-UAVs based on Apollonius circle in the environment with obstacle,” *Connection Science*, 2023.
- [6] “Area-Optimal Control Strategies for Heterogeneous Multi-Agent Pursuit,” arXiv:2511.15036, 2025.
- [7] “A comprehensive survey of weapon target assignment problem: Model, algorithm, and application,” *Engineering Applications of Artificial Intelligence*, 2024.
- [8] H.-L. Choi, L. Brunet, and J. P. How, “Consensus-Based Decentralized Auctions for Robust Task Allocation,” *IEEE Trans. Robotics*, vol. 25, no. 4, 2009.
- [9] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, pp. 83–97, 1955.
- [10] D. Reid, “An Algorithm for Tracking Multiple Targets,” *IEEE Trans. Automatic Control*, vol. 24, no. 6, pp. 843–854, 1979.
- [11] B.-N. Vo and W.-K. Ma, “The Gaussian Mixture Probability Hypothesis Density Filter,” *IEEE Trans. Signal Processing*, vol. 54, no. 11, pp. 4091–4104, 2006.
- [12] S. Reuter, B.-T. Vo, B.-N. Vo, and K. Dietmayer, “The Labeled Multi-Bernoulli Filter,” *IEEE Trans. Signal Processing*, vol. 62, no. 12, 2014.
- [13] H. A. P. Blom and Y. Bar-Shalom, “The IMM algorithm for systems with Markovian switching coefficients,” *IEEE Trans. Automatic Control*, vol. 33, no. 8, 1988.
- [14] Defence Science and Technology Laboratory (dstl), “Stone Soup: an open-source framework for tracking and state estimation,” 2017.
- [15] M. Rahman and D. A. Robertson, “Radar micro-Doppler signatures of drones and birds at K-band and W-band,” *Scientific Reports*, vol. 8, 2018.
- [16] “RF-Enabled Deep-Learning-Assisted Drone Detection and Identification: An End-to-End Approach,” *Sensors*, vol. 23, no. 9, 4202, 2023.
- [17] Open-source intelligence on the Gerbera decoy drone (ISIS report; CEPA; Army Recognition analysis), 2024–2025.
- [18] P. Zarchan, *Tactical and Strategic Missile Guidance*, 6th/7th ed. AIAA Progress in Astronautics and Aeronautics, 2012/2019.
- [19] M. Vrba et al. (CTU MRS), “Autonomous capture of agile flying objects using UAVs: the MBZIRC 2020 challenge,” *Robotics and Autonomous Systems*, 2022.
- [20] JARUS, “Guidelines on Specific Operations Risk Assessment (SORA),” JAR-DEL-WG6-D.04.
- [21] S. Primatesta, A. Rizzo, and A. la Cour-Harbo, “Ground Risk Map for Unmanned Aircraft in Urban Environments,” *J. Intelligent & Robotic Systems*, 2020.
- [22] “Accurate Ground Impact Footprints and Probabilistic Maps for Risk Analysis of UAV Missions,” *IEEE Aerospace Conference*, 2019.
- [23] R. Lowe et al., “Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments” (MADDPG), NeurIPS 2017; C. Yu et al., “The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games” (MAPPO), NeurIPS 2022.
- [24] M. Fernandez-Cortizas et al., “Aerostack2: A Software Framework for Developing Multi-robot Aerial Systems,” arXiv:2303.18237, 2023.

Reproducibility. This document and its figures are generated by `scripts/build_report.py` and `scripts/report_figures.py` from the COOP-UAV-S repository. The quantitative results reproduce the documented reference Monte-Carlo (README.md; docs/ARCHITECTURE.md §4) and the diving-jet acquisition latency (docs/MARL.md). Several references list standard bibliographic details that should be re-verified against the primary source before formal external citation.